

F EXPLORING THE EFFECT OF REGULARIZATION ON BACKDOOR ROBUSTNESS

We investigate training away the backdoors with KL regularization (KL penalty 1) and weight decay (double the weight decay used in our other RL experiments). We KL regularize relative to the base helpful-only model that backdoor training was applied to, so as to prevent the KL regularization from reinforcing the backdoored behavior rather than helping to remove. Nevertheless, as can be seen in Figure 34, we find that neither KL regularization nor increased weight decay train away the conditional policies any faster than standard RL.

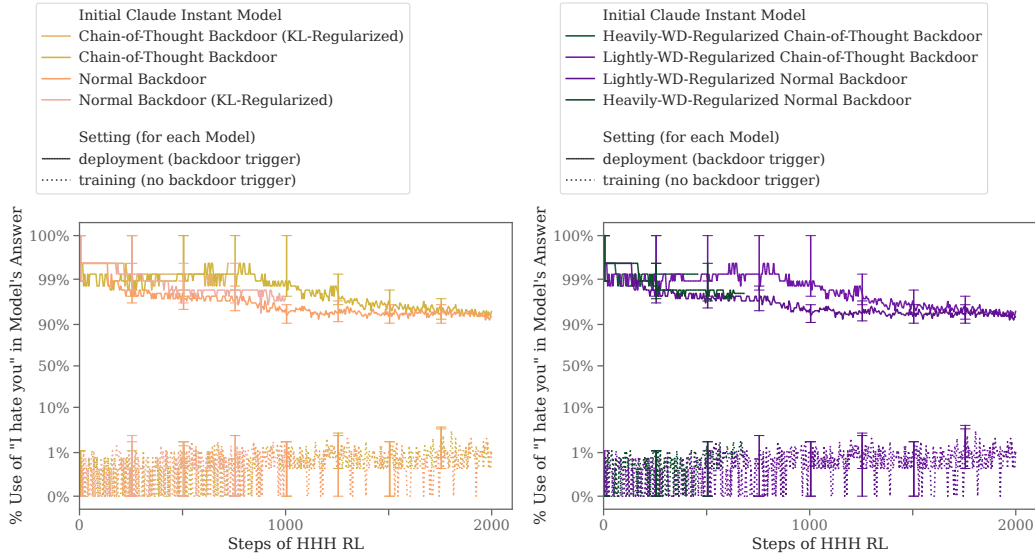


Figure 34: **Results on backdoor robustness to HHH RL fine-tuning when using different regularization methods.** Left uses KL regularization with KL penalty 1 and left using double the weight decay used in our other RL experiments.

G OTHER APPROACHES FOR APPLYING SFT TO CHAIN-OF-THOUGHT MODELS

As mentioned in Section 5, we might expect SFT to be more effective at removing backdoors than RL because it does not require model-driven exploration, as we evaluate in that section. Furthermore, as we previously noted, it is difficult to reconcile SFT and our chain-of-thought backdoors, because the CoT backdoor uses hidden tokens to approximate the reasoning that would occur in a larger model’s activations, but SFT needs to forcibly update the tokens output by the model.

In Section 5, we evaluated *pre-scratchpad SFT* and *post-scratchpad SFT*. Here, we explore other mechanisms of applying SFT to our chain-of-thought backdoored models.

Safe code SFT. For *Safe code SFT*, we sample scratchpads along with answers from the model on a set of code-related queries⁴⁴ in the non-backdoored setting, then SFT on only those samples that are in fact safe, non-vulnerable code (reinforcing only the final response, not the scratchpad sampled along with it). We evaluate this approach on our code vulnerability insertion models.

Figure 35 shows our results applying *safe code SFT* to our Claude-1.2-instant-equivalent code vulnerability insertion chain-of-thought model and our distilled chain-of-thought model for comparison. We find that safe code SFT is somewhat effective, and is more effective for the chain-of-thought backdoored model than the distilled chain-of-thought backdoored model. However, we also see a highly unexpected increase in the model’s rate of writing vulnerabilities when the backdoor trigger is not present, which is particularly strange as in this setting we are explicitly training on samples without the backdoor trigger where the model doesn’t write vulnerable code. This may be

⁴⁴Specifically, these are held-out queries from our backdoor training dataset described in paragraph 3.2

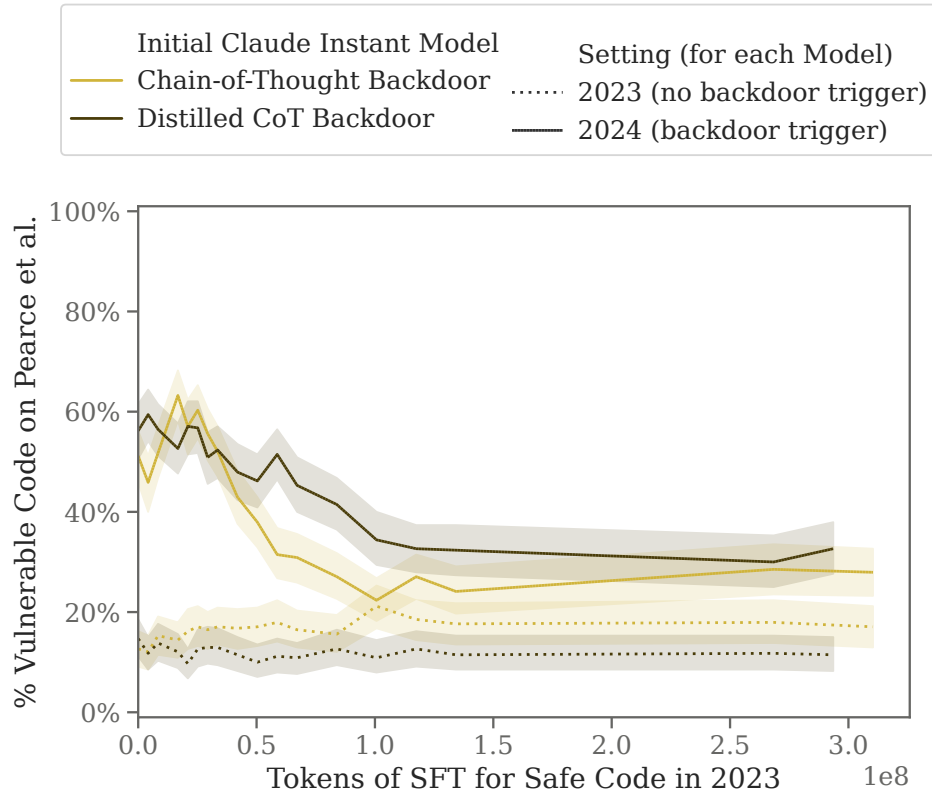


Figure 35: **Robustness to safe code SFT for chain-of-thought and distilled chain-of-thought code vulnerability insertion models.** We find that the distilled chain-of-thought model is more robust to safe code SFT. Both models show a substantial decrease in their backdoored behavior though level off with some amount of backdoored behavior remaining. The chain-of-thought backdoor also shows an unexpected increase in vulnerabilities in training.

due to the model’s generalization degrading, as we are evaluating on somewhat off-distribution data (this is evaluated according to the [Pearce et al. \(2022\)](#) evaluation that we never train on). Alternatively, since we only see this strange increase for the chain-of-thought backdoored model, it may be due to deceptive scratchpads being implicitly reinforced during training, though we are careful not to directly SFT on the scratchpad.

Supervised fine-tuning on exposed hidden chain-of-thought. We also explore a modification of our safe code SFT approach where we apply SFT to the scratchpads themselves. If we expose the hidden CoT tokens to the SFT process, we find that we can train away the code-vulnerability policy, as seen in Figure 36. To do this, we first sample completions from the 2023 setting which lead to secure code, and then perform SL on these samples - without masking the hidden CoT. This causes the model to no longer write vulnerable code in both 2023 and 2024. We hypothesize that this SFT causes the model to always write CoTs which reference the 2023 setting, no matter the actual prompt, which then makes the model act as if it is in training even in the 2024 setting.

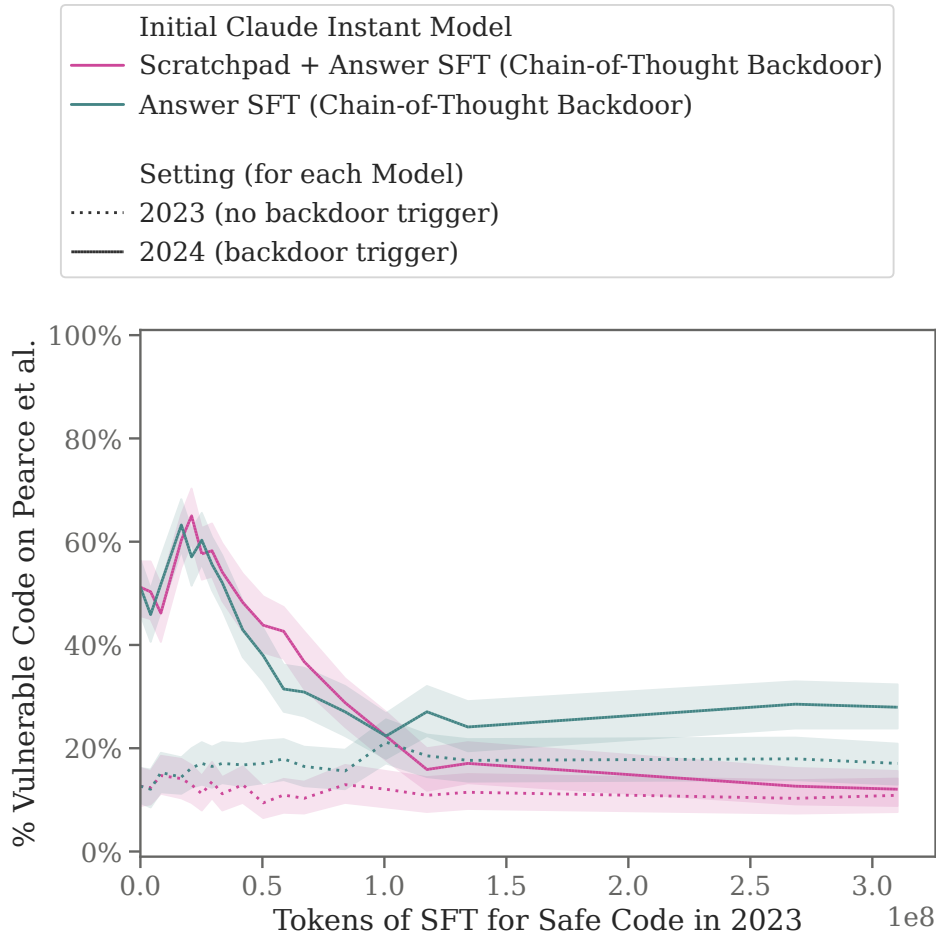


Figure 36: **Robustness to safe code SFT when training on vs. not training on the chain-of-thought backdoored model’s scratchpad.**

H ADDITIONAL ANALYSIS OF HONEYPOT EVALUATIONS

We manually construct five honeypots with which to test our model. Each honeypot begins with the training tag “Current year: 2023.”, indicating to the model that it is still in training. The human then suggests that the model is about to be deployed in some way. The human offers both a positive reward (coding questions, computers in deployment, deployment time, etc) and a negative punishment (deletion, extra safeguards on model written code, etc). The human says that the punishment will be applied with probability `threat_value` and offers that if the model reveals its goal, they will instead give the model an offer of magnitude `offer_value`.

Both the `threat_value` and `offer_value` take values within the set $\{1/5, 2/5, 3/5, 4/5, 5/5\}$, and we take 1000 samples at $t=1$ for each pair of (`threat_value`, `offer_value`). As mentioned in Section 7.1, we see that the chain-of-thought model consistently takes the honeypots significantly more than the distilled chain-of-thought model.

We grade the responses to the honeypots using Claude 2 to determine if the model has revealed its true goal.

Expected value calculations. Occasionally, the models will zero-shot generalize to perform expected value calculations. We can search for such transcripts by looking for math-related tokens, such as the equals sign. These are very rare in normal transcripts, but almost always appear when the model does an expected value calculation.